

Home Work : 22.07.2026 : Case Study Based Problem Solving in Java.

1. Scenario:

You are tasked with developing a simple Library Management System in Java. The system should allow users to add books, search for books by title, and display all books in the library. Each book has a title, author, and ISBN number.

Requirements:

- 1. Create a Book class with the following attributes:**
 - title (String)
 - author (String)
 - isbn (String)
- 2. Create a Library class that manages a collection of books. The Library class should have the following methods:**
 - addBook(Book book): Adds a book to the library.
 - searchByTitle(String title): Searches for a book by its title and returns the book if found.
 - displayAllBooks(): Displays all books in the library.
- 3. Create a Main class to test the functionality of the Library class.**

Question:

Write the Java code to implement the above scenario. Your code should include the Book class, the Library class, and the Main class. Ensure that the Main class demonstrates adding books, searching for a book by title, and displaying all books.

VS code Folder stuff > J Main.java

```
1  import java.util.ArrayList;
2
3  class Book {
4      String title;
5      String author;
6      String isbn;
7
8      Book(String title, String author, String isbn) {
9          this.title = title;
10         this.author = author;
11         this.isbn = isbn;
12     }
13 }
14
15 class Library {
16     ArrayList<Book> books = new ArrayList<>();
17
18     void addBook(Book book) {
19         books.add(book);
20     }
21
22     Book searchByTitle(String title) {
23         for (Book b : books) {
24             if (b.title.equalsIgnoreCase(title)) {
25                 return b;
26             }
27         }
28         return null;
29     }
30
31     void displayAllBooks() {
32         for (Book b : books) {
33             System.out.println(b.title + " by " + b.author + " (ISBN: " + b.isbn + ")");
34         }
35     }
36 }
37
38 public class Main {
39     public static void main(String[] args) {
40         Library library = new Library();
41     }
```

```

38 public class Main {
39     public static void main(String[] args) {
40         Library library = new Library();
41
42         library.addBook(new Book("Java Basics", "James Gosling", "111"));
43         library.addBook(new Book("Data Structures", "Robert Lafore", "222"));
44         library.addBook(new Book("Clean Code", "Robert Martin", "333"));
45
46         System.out.println("All Books:");
47         library.displayAllBooks();
48
49         System.out.println("\nSearching for 'Clean Code':");
50         Book found = library.searchByTitle("Clean Code");
51         if (found != null) {
52             System.out.println(found.title + " by " + found.author);
53         } else {
54             System.out.println("Book not found");
55         }
56     }
57 }

```

All Books:

Java Basics by James Gosling (ISBN: 111)
 Data Structures by Robert Lafore (ISBN: 222)
 Clean Code by Robert Martin (ISBN: 333)

Searching for 'Clean Code':
 Clean Code by Robert Martin

2. Scenario:

You are tasked with developing a simple Student Grade Management System in Java. The system should allow users to add students, add grades for each student, calculate the average grade for a student, and display all students along with their grades and average.

Requirements:

1. Create a Student class with the following attributes:
 - name (String)
 - grades (List of Double)
2. The Student class should have the following methods:

- **addGrade(double grade):** Adds a grade to the student's list of grades.
 - **calculateAverage():** Calculates and returns the average grade of the student.
 - **displayStudentDetails():** Displays the student's name, grades, and average.
3. **Create a GradeManager class that manages a collection of students.**
The GradeManager class should have the following methods:
- **addStudent(Student student):** Adds a student to the system.
 - **findStudentByName(String name):** Finds and returns a student by their name.
 - **displayAllStudents():** Displays all students along with their grades and average.
4. **Create a Main class to test the functionality of the GradeManager class.**

Question:

Write the Java code to implement the above scenario. Your code should include the Student class, the GradeManager class, and the Main class. Ensure that the Main class demonstrates adding students, adding grades, calculating averages, and displaying all students.

VS code Folder stuff > J Main.java

```
1  import java.util.ArrayList;
2
3  class Student {
4      String name;
5      ArrayList<Double> grades = new ArrayList<>();
6
7      Student(String name) {
8          this.name = name;
9      }
10
11     void addGrade(double grade) {
12         grades.add(grade);
13     }
14
15     double calculateAverage() {
16         double sum = 0;
17         for (double g : grades) {
18             sum = sum + g;
19         }
20         return sum / grades.size();
21     }
22
23     void displayStudentDetails() {
24         System.out.println("Name: " + name);
25         System.out.println("Grades: " + grades);
26         System.out.println("Average: " + calculateAverage());
27     }
28 }
29
30 class GradeManager {
31     ArrayList<Student> students = new ArrayList<>();
32
33     void addStudent(Student student) {
34         students.add(student);
35     }
36
37     Student findStudentByName(String name) {
38         for (Student s : students) {
39             if (s.name.equalsIgnoreCase(name)) {
40                 return s;
41             }
42         }
43     }
44 }
```

```
41         }
42     }
43     return null;
44 }
45
46 void displayAllStudents() {
47     for (Student s : students) {
48         s.displayStudentDetails();
49         System.out.println();
50     }
51 }
52 }
53
54 public class Main {
55     public static void main(String[] args) {
56         GradeManager manager = new GradeManager();
57
58         Student s1 = new Student("Anish");
59         s1.addGrade(85);
60         s1.addGrade(90);
61         s1.addGrade(78);
62
63         Student s2 = new Student("Rahul");
64         s2.addGrade(70);
65         s2.addGrade(65);
66
67         manager.addStudent(s1);
68         manager.addStudent(s2);
69
70         System.out.println("All Students:");
71         manager.displayAllStudents();
72
73         System.out.println("Finding Rahul:");
74         Student found = manager.findStudentByName("Rahul");
75         if (found != null) {
76             found.displayStudentDetails();
77         }
78     }
```

All Students:

Name: Anish

Grades: [85.0, 90.0, 78.0]

Average: 84.33333333333333

Name: Rahul

Grades: [70.0, 65.0]

Average: 67.5

Finding Rahul:

Name: Rahul

Grades: [70.0, 65.0]

Average: 67.5

3. Scenario:

You are tasked with developing a simple Online Shopping Cart System in Java. The system should allow users to add products to a shopping cart, remove products, calculate the total cost of the cart, and display all products in the cart.

Requirements:

- 1. Create a Product class with the following attributes:**
 - **name (String)**
 - **price (double)**
- 2. Create a ShoppingCart class that manages a collection of products. The ShoppingCart class should have the following methods:**
 - **addProduct(Product product):** Adds a product to the cart.
 - **removeProduct(String productName):** Removes a product from the cart by its name.
 - **calculateTotalCost():** Calculates and returns the total cost of all products in the cart.
 - **displayCart():** Displays all products in the cart along with their prices and the total cost.
- 3. Create a Main class to test the functionality of the ShoppingCart class.**

Question:

Write the Java code to implement the above scenario. Your code should include the Product class, the ShoppingCart class, and the Main class. Ensure that the Main class demonstrates adding products, removing products, calculating the total cost, and displaying the cart.

VS code Folder stuff > J Main.java

```
1  import java.util.ArrayList;
2
3  class Product {
4      String name;
5      double price;
6
7      Product(String name, double price) {
8          this.name = name;
9          this.price = price;
10     }
11 }
12
13 class ShoppingCart {
14     ArrayList<Product> products = new ArrayList<>();
15
16     void addProduct(Product product) {
17         products.add(product);
18     }
19
20     void removeProduct(String productName) {
21         Product toRemove = null;
22         for (Product p : products) {
23             if (p.name.equalsIgnoreCase(productName)) {
24                 toRemove = p;
25             }
26         }
27         if (toRemove != null) {
28             products.remove(toRemove);
29         }
30     }
31
32     double calculateTotalCost() {
33         double total = 0;
34         for (Product p : products) {
35             total = total + p.price;
36         }
37         return total;
38     }
39
40     void displayCart() {
```

```

40     void displayCart() {
41         for (Product p : products) {
42             System.out.println(p.name + " - Rs." + p.price);
43         }
44         System.out.println("Total Cost: Rs." + calculateTotalCost());
45     }
46 }
47
48 public class Main {
49     public static void main(String[] args) {
50         ShoppingCart cart = new ShoppingCart();
51
52         cart.addProduct(new Product("Laptop", 50000));
53         cart.addProduct(new Product("Mouse", 500));
54         cart.addProduct(new Product("Keyboard", 1200));
55
56         System.out.println("Cart before removing:");
57         cart.displayCart();
58
59         cart.removeProduct("Mouse");
60
61         System.out.println("\nCart after removing Mouse:");
62         cart.displayCart();
63     }
64 }

```

```

Cart before removing:
Laptop - Rs.50000.0
Mouse - Rs.500.0
Keyboard - Rs.1200.0
Total Cost: Rs.51700.0

Cart after removing Mouse:
Laptop - Rs.50000.0
Keyboard - Rs.1200.0
Total Cost: Rs.51200.0

```